

ARTEFACT DOCUMENTATION

Automated Benchmarking Framework for Network Emulation Platform Comparison

Item	Details
Project	Deploying Secure Distributed Systems: Comparative Analysis of Mininet and SEED Internet Emulator
Platforms Deployed	Mininet (FRRouting, OSPF), SEED Internet Emulator (BIRD, OSPF).
Artefact Type	Python benchmarking framework, topology implementations, OSPF routing configuration, raw result files and documentation.
Author	Emmanuel Odianose Aghedo
Degree	MSc Cybersecurity – Teesside University
Academic Year	2025 - 2026
Repository	https://github.com/AghedoEmmanuel/network-emulation-benchmark

This document is submitted as part of the MSc Computing artefact submission. It explains the design, implementation, benchmarking methodology and experimental results underpinning the dissertation titled “A Comparative Performance Evaluation of Network Emulation Platforms (Mininet vs SEED Internet Emulator) Using Automated Python-Based Benchmarking.”

1. Artefact Overview

This artefact is a complete automated benchmarking framework designed to compare two network emulation environments under controlled and secured experimental conditions. It offers the benchmarking scripts, raw result datasets, statistical comparison scripts, OSPF routing configuration files, topology implementation scripts, and supporting documentation needed for the dissertation research.

A six-node ring topology that is implemented similarly across Mininet with FRRouting (FRR) and the SEED Internet Emulator with BIRD is the basis for the comparison. Performance disparities can be more confidently ascribed to the underlying emulator design and routing implementation rather than topological asymmetry when both platforms maintain a consistent logical topology.

The artefact was designed with reproducibility as a primary concern. All test scripts are automated and produce structured CSV and JSON output files so that results can be independently verified.

2. Research Aim and Objectives

The aim of this study is to use automated benchmarking and analogous OSPF-based routing designs to assess and compare the network performance characteristics of Mininet and SEED Internet Emulator under controlled emulation settings.

The objectives are:

- Create a six-node ring topology that works in both emulation settings.
- Implement OSPF using FRRouting (FRR) dynamic routing in Mininet.
- Implement OSPF using BIRD dynamic routing in SEED IE.
- To get performance data with the least amount of human interaction, thereby creating an automated Python benchmarking program.
- Collect a statistically meaningful number of repeated measurements.
- Export raw results to structured CSV and JSON files for transparency and reproducibility.
- Examine the differences between the two systems' latency, throughput, bandwidth, jitter, packet loss, convergence time, CPU consumption, and memory usage.
- Examine the latency of inter-emulator communication between the two environments.
- Provide a reproducible benchmarking artefact suitable for academic evaluation and viva demonstration.

3. Relationship to the Dissertation

The dissertation's empirical chapters are directly supported by this artefact. The raw CSV and JSON files included in the result folder are the source of all performance tables and statistical results presented in Chapter 5 (Performance Comparison). The path from raw data to reported findings is entirely traceable since the comparison scripts (`comparison.py`, `convergence_comparison.py`, `final_comparison_script.py`) replicate the summary statistics used in the dissertation tables.

The methodology implemented here follows the six-step benchmarking process outlined in the dissertation's Methodology chapter: topology construction, interface configuration, OSPF convergence verification, test execution, data collection, and statistical analysis. The artefact demonstrates that this process was applied consistently to both platforms, supporting the validity of the comparative analysis.

The experimental design is informed by Golightly, Modesti and Chang (2023), who conducted a comparable evaluation across GNS3 and SEED Internet Emulator. This artefact mirrors that paper's topology-matched comparative approach.

4. Repository and Artefact Structure

The artefact is submitted as a zipped folder containing the full GitHub repository, raw result files, topology diagrams, screenshots, and this artefact documentation document. Submitting a fixed zip rather than only a GitHub link ensures that the marker receives the exact version assessed, with no risk of post-submission changes.

Folder / File	Purpose
README.md	Quick-start overview: setup, metrics, run instructions.
docs/diagrams/	Topology diagrams and visual documentation.
mininet_ospf/	Mininet FRR/OSPF topology script, FRR configuration files, benchmark scripts.
mininet_ospf_hybrid/	Mininet files used in hybrid inter-emulator testing.
seed_ie/	SEED IE topology and OSPF setup scripts; hybrid variant.
hybrid_system_comparison/	Scripts for inter-emulator latency testing between Mininet and SEED IE.
result/	All raw and processed benchmark CSV/JSON outputs.
comparison.py	Generates comparison-ready summary from raw result files.
convergence_comparison.py	Analyses failure and recovery convergence data.
final_comparison_script.py	Produces final summary tables used in the dissertation.
inter_latency.py	Measures latency between the two emulated environments.
latency_table.py	Formats latency results into table form for analysis.

mininet.py / seed.py	Standalone platform-level benchmark entry points.
----------------------	---

5. Topology Design

The same six-node ring architecture is used in both settings. The benchmarking study may evaluate both neighbouring one-hop routes and non-adjacent two-hop routes because the topology supports multiple path lengths. It is feasible to investigate if performance differences between the two platforms are continuous over route distances or just appear at specific path lengths by including numerous path lengths, which is methodologically significant.

Route Pair	Hop Count	Reason for Inclusion
R1 → R2	1	Adjacent route; baseline one-hop latency and throughput.
R1 → R3	2	Two-hop path; tests forwarding through an intermediate router.
R3 → R2	1	Adjacent route from a different ring position.
R3 → R6	2	Two-hop route across the ring; asymmetric position test.
R1 → R6	1	Adjacent on the opposing side of the ring.
R2 → R5	2	Longer logical route for cross-ring performance comparison.
R4 → R5	1	Adjacent pair in the lower segment of the ring.
R4 → R6	2	Two-hop; tests path variation within the lower ring segment.
R6 → R1	1	Reverse-direction adjacent route; checks bidirectional consistency.

6. Mininet FRR/OSPF Implementation

The Mininet implementation uses virtual Ethernet cables and Linux network namespaces to build the six-router architecture. Here, Mininet is set up as a routed testbed instead of functioning as an SDN switching environment controlled by a centralised controller. To enable OSPF dynamic routing, FRRouting (FRR) is installed on every router node.

6.1 Configuration Approach

- The topology script allocates point-to-point virtual Ethernet connections that correspond to the ring topology design and builds router-style nodes.
- Each router interface is assigned a /30 IP address from the 10.0.x.0/30 address space and from the 10.1.x.0/30 address space for the hybrid.
- To enable inter-interface packet forwarding, Linux IP forwarding is enabled on every node.
- FRR is configured with OSPF Area 0 on all router interfaces. Connected networks are advertised automatically.

- OSPF neighbour relationships are verified before benchmarking begins to ensure all routes are converged
- Benchmark scripts gather per-iteration latency, throughput, jitter, convergence, and resource measures using the defined OSPF pathways.

6.2 Evidence Included

- Mininet topology script (mn_topo.py for mininet_ospf and Mininet_ospf_hybrid).
- FRR configuration files (zebra.conf, ospfd.conf) for each router node.
- A config_frr.sh file to run the frr configuration in mn_topo.py.
- Screenshot or text output showing OSPF neighbour and route table entries.
- The result directory contains raw benchmark CSV/JSON output files

7. SEED Internet Emulator Implementation

The SEED Internet Emulator (SEED IE) creates the same logical six-node ring topology using Docker containers. Each router is a separate container running BIRD, and point-to-point networks between containers reproduce the ring structure. SEED IE is not a hypervisor-based system; it uses Docker to provide lightweight, isolated router instances that can be built and started reproducibly from Python configuration scripts.

7.1 Configuration Approach

- The SEED IE framework API (SEED_Ring.py) is used to define the SEED IE topology in Python.
- Router nodes are assigned to a single autonomous system (AS100) with point-to-point /30 subnets.
- OSPF is enabled on all router interfaces so that dynamic route discovery operates within the emulated environment.
- The generated Docker Compose environment is built and started before benchmarking begins.
- Benchmark scripts enter each container (or connect to it remotely) to run tests and collect measurements.
- Results are exported to CSV and JSON for analysis.

7.2 Evidence Included

- The SEED_Ring.py SEED IE topology script is in seed_ie/.
- The SEED_ring_hybrid.py SEED IE topology script is in seed_ie/.
- A screenshot displaying the names of the Docker containers that are now executing.
- Screenshot or terminal output showing OSPF routing table entries within a container.
- The result directory contains raw benchmark CSV/JSON output files.

8. Benchmarking Scripts and Metrics

To minimise human error and guarantee repeatability, all benchmarking is automated using Python scripts. Each script outputs data to CSV and JSON, maintains individual iteration values, and performs an adjustable number of repeated measurements. Instead of being manually approximated, summary statistics (minimum, maximum, mean, median, and standard deviation) are calculated from the raw data.

Script Category	Function
Latency testing	Runs repeated ICMP ping tests; records individual RTT values per route pair.
Jitter processing	Computes delay variation across repeated latency samples.
Throughput / bandwidth	Uses iperf3 to measure actual data transfer rate across each tested path.
Packet loss	Records the proportion of probes that fail to reach the destination.
Convergence testing	Disables a link, records time to alternate-path activation (failure convergence); restores the link, records time to full reachability (recovery convergence).
Resource overhead	Reads /proc/stat and /proc/meminfo to record CPU and memory utilisation during test execution.
Inter-emulator testing	Measures latency between Mininet router nodes and SEED IE container nodes via a bridged network interface.
Comparison / analysis	Converts raw result files into summary tables and descriptive statistics for dissertation chapters.

Metrics Collected:

Metric	Definition	Unit
Latency	Round-trip delay measured by ICMP echo request/reply.	ms
Jitter	Standard deviation of RTT values across repeated samples.	ms
Throughput	Actual data transfer rate achieved during iperf3 testing.	Mbps
Bandwidth	Maximum measured transfer capacity of the tested path.	Mbps
Packet Loss	Percentage of probes that did not receive a reply.	%
Convergence Time (Failure)	Time from link failure to routing via alternate path.	ms
Convergence Time (Recovery)	Time from link restoration to full reachability.	ms
CPU Utilisation	Host CPU usage sampled from /proc/stat.	%

Memory Utilisation	Host memory in use sampled from /proc/meminfo.	%
Inter-Emulator Latency	RTT between a Mininet node and a SEED IE container node.	ms

9. Benchmarking Methodology

The benchmarking process follows a structured sequence applied consistently to both platforms. To be sure that observed discrepancies are due to platform behaviour rather than procedural inconsistency, the identical sequence must be applied to each environment.

- Step 1: Create a ring topology with six nodes and activate every interface.
- Step 2: Assign IP addresses to all router interfaces in accordance with the addressing plan.
- Step 3: Allow the protocol to converge after starting OSPF routing on every router node.
- Step 4: Verify full reachability between all router pairs before any benchmarking begins.
- Step 5: Execute benchmark scripts for steady-state metrics (latency, throughput, jitter, packet loss, resource overhead). Each metric is collected over $n = 100$ iterations per route pair.
- Step 6: Convergence testing should be carried out separately. After inducing a connection failure, the convergence time is noted, the link is restored, and the recovery convergence time is noted.
- Step 7: Export all raw per-iteration values to CSV or JSON files or both.
- Step 8: Run comparison and analysis scripts to generate summary statistics and dissertation tables.

10. Result Files

All the raw benchmark output files created during testing are in the result/folder. It is a purposeful design decision to keep raw per-iteration data instead of just summary statistics because it enables independent reproduction of the dissertation tables, permits further statistical analysis if necessary, and shows that results were gathered empirically rather than estimated.

11. How to Run the Artefact

The instructions below describe how to reproduce the experimental environment. Network emulation scripts require root/administrator privileges because they create virtual network interfaces, network namespaces, and Docker containers.

11.1 Clone the Repository

```
git clone https://github.com/AghedoEmmanuel/network-emulation-benchmark.git  
  
cd  
  
network-emulation-benchmark
```

11.2 Install Common Dependencies

```
sudo apt update  
  
sudo apt install -y python3 python3-pip iputils-ping iperf3 docker.io docker-compose  
  
sudo systemctl enable docker  
  
sudo systemctl start docker
```

11.3 Run the Mininet OSPF Environment

11.3.1 Clone Mininet:

```
git clone https://github.com/mininet/mininet.git  
  
cd mininet
```

11.3.2 Move the Mininet OSPF and Mininet OSPF hybrid folder into the mininet folder:

```
mv [source_folder] [destination_folder]
```

Replace [source_folder] with the actual destination of the mininet OSPF folder.

Replace [destination_folder] with the actual destination of the mininet folder.

11.3.3 Move into the Mininet OSPF folder:

```
cd mininet/mininet_ospf or cd mininet/mininet_ospf_hybrid
```


11.3.4 Install Mininet Dependencies:

```
sudo util/install.sh -n
```

11.3.5 Make config_frr.sh executable:

```
chmod +x config_frr.sh
```

11.3.6 Run config_frr.sh:

```
sudo ./config_frr.sh
```

11.3.7 Run the Mininet topology script:

```
sudo python3 mn_topo.py
```

Confirm that the FRR configuration files are in the expected location before running the benchmark.

11.3.8 Run Comparison Scripts

From the project root:

```
sudo python3 newbenchmark_mininet.py
```

```
sudo python3 benchmark_mininet_tbcm.py
```

11.4 Run the SEED IE Environment

11.4.1 Clone SEED IE:

```
git clone https://github.com/seed-labs/seed-emulator.git
```

```
cd seed-emulator
```

11.4.2 Move the seed_ie folder into the seed-emulator examples folder:

```
mv [source_folder] [destination folder]
```

Replace [source_folder] with the actual destination of the seed_ie folder.

Replace [destination folder] with the actual destination of the seed-emulator/examples folder.

In this implementation, the SEED IE scripts should be run from the examples directory of the SEED Emulator installation.

11.4.3 Move into the SEED IE folder:

```
cd seed-emulator
```

11.4.4 Install a Python Virtual Environment:

```
sudo python3 -m venv venv
```

11.4.5 Install your requirements inside the virtual environment:

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

```
source development.env
```

11.4.6 Move into the seed_ie folder:

```
cd examples/seed_ie
```

11.4.7 Run the SEED IE topology script:

```
python3 SEED_Ring.py
```

and for the hybrid layout

```
python3 SEED_ring_hybrid.py
```

This will produce an output folder.

11.4.8 Move into the output folder:

```
cd output
```

Depending on the SEED setup, the generated Docker environment may then need to be built and started using Docker Compose.

```
Sudo docker-compose build
```

Sudo docker-compose up

11.4.9 Run Comparison Scripts

From the project root:

```
python3 newbenchmark_seed.py
```

```
python3 benchmark_seed_tbcm.py
```

11.5 Run Inter-Emulator Latency Testing

```
python3 inter_latency.py
```

This script is used to test latency between Mininet and SEED IE when both environments are connected for hybrid/inter-emulator testing.